HiTrace Solutions — LoanNBFC (Gidfine)

Phase 5 — Target Architecture Reassessment & FastAPI Decommissioning Plan | Date: October 17, 2026 | Branch: development

1. Executive Summary & Directive Reassessment

Following the supervisor instruction (**"We won't use FastAPI"**), Phase 5 conducts a comprehensive reassessment of the entire backend stack, database tier, and service architecture. The project is transitioning from Vue to React, with **Convex** as the primary candidate backend and data layer. This assessment inventories all 72 active FastAPI endpoints, maps underlying business logic, establishes Convex readiness, and presents a non-disruptive, ordered migration plan without premature deletion or unconfirmed technology choices.

Git Branch: development (Protected)	**Directive:** Decommission FastAPI	**Candidate Backend:** Convex
Endpoints Inventoried: 72 Endpoints	**Core Collections:** 5 MongoDB Collections	**Action Taken:** Analysis & Planning Only

2. Backend Capability Inventory & Target Destination

Domain / Capability	FastAPI Endpoint Count	Target Destination in Convex	Migration Feasibility & Notes
Auth & Profile	14 endpoints	Convex Auth / Clerk + users table	Direct. Convex Auth manages sessions, JWTs, and profile mutations cleanly.
Session State	12 endpoints	Convex sessions table + Live Subscriptions	Direct improvement. Replaces REST polling + custom WebSockets with reactive queries.
Comparison & What-If	4 endpoints	Convex TypeScript module (convex/comparison.ts)	Direct. Phase 4 scoring, ranker algorithms, and schemas port 100% to TypeScript.
Underwriting & SHAP	4 endpoints	Convex internal mutations (convex/underwrite.ts)	Direct. Mathematical scoring (DTI, FOIR, benchmark spreads, SHAP) is deterministic.
Document OCR & Fraud	8 endpoints	Convex File Storage + Convex Action (Vision LLM)	Action-based. Binary uploaded to Convex storage; Action calls Vision API.
Sanction PDF & E-Sign	4 endpoints	Convex Action (Node pdf-lib) + loans table	Requires Decision. ReportLab is Python; migrate to Node PDF engine or external service.
Repayments & Razorpay	5 endpoints	Convex httpAction (Webhooks) + payments table	Direct. Convex HTTP actions handle order creation & signature verification.
Admin & Analytics	13 endpoints	Convex Index Queries & Aggregations	Direct. Conversion funnel, lender stats, and KPIs run natively on Convex tables.
AI Advisor Chat	8 endpoints	Convex chat_messages table + Action (LLM)	Requires Decision. Replace LangGraph graph with Convex Action or keep Python microservice.

3. Proposed Convex Schema & Security Model

• **Proposed Tables:**

1. users: Indexed by phone & email. Stores borrower KYC profiles, pre-approved limits, hashed creds.
2. sessions: Tracks origination phase, requested loan parameters, selected lender, underwriting flags.
3. documents: References Convex _storage ID, document type, extracted OCR values, tamper check booleans.
4. chat_messages: Indexed by sessionId. Real-time streaming conversational history between borrower & AI Advisor.
5. lenders: Active Ghanaian banking partners, interest rate ranges, FOIR ceilings, and approval probabilities.
6. loans: Disbursed loan contracts, principal, tenure, EMI, first payment date, and active ledger status.
7. payments: EMI transaction log, Razorpay payment ID, timestamp, and remaining balance.

• **Security & Zero-Exposure Principle:** Sensitive documents (National IDs, salary slips) are accessed only via temporary signed URLs. Underwriting algorithms run strictly inside server-side Convex mutations; client callers cannot tamper with interest rates or credit scores.

4. Recommended Phased Implementation Sequence

Phase 6 (Convex Foundation): Initialize Convex, install convex in frontend/package.json, implement convex/schema.ts, and configure ConvexProvider in React.
Phase 7 (Core Logic Porting): Translate financial_rules.py, loan_ranker.py, and comparison.py to TypeScript in convex/lib/ with comprehensive Vitest tests verifying parity against Phase 4.
Phase 8 (Reactive Frontend Integration): Replace lib/api.ts REST calls with Convex useQuery and useMutation hooks. Decommission custom WebSocket client in favor of native Convex subscriptions.
Phase 9 (Document & PDF Resolution): Implement Convex File Storage for uploads; integrate Node PDF generation (or approved service).
Phase 10 (FastAPI Deprecation & Codebase Pruning): Verify end-to-end functionality on React + Convex, archive .vue files, and decommission Python FastAPI codebase on supervisor approval.

5. Critical Decisions for Supervisor Clarification

1. **AI Orchestration / LangGraph:** Python LangGraph currently manages multi-agent handoffs. Should we migrate agent workflows to Convex Actions (calling Gemini directly), or retain an isolated microservice?
2. **Sanction PDF Generation:** Python ReportLab currently generates legal sanction PDFs. Shall we adopt a Node-based engine (e.g. pdf-lib or @react-pdf/renderer) within Convex Actions?
3. **Authentication Provider:** Does the supervisor prefer Convex Auth, Clerk, or phone/OTP custom authentication inside Convex?